

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА

Факультет комп'ютерних наук та кібернетики
Кафедра інтелектуальних програмних систем

Курсова робота

за спеціальністю 121 Інженерія програмного забезпечення
на тему:

РОЗРОБКА МОБІЛЬНОГО ЗАСТОСУНКУ ДЛЯ МОНІТОРИНГУ
ТА УПРАВЛІННЯ ЖИТТЄВИМ ЦИКЛОМ КОСМЕТИЧНИХ ЗАСОБІВ

Виконала студентка 3-го курсу
ОПП «Програмна інженерія»
Клевчук Марія Вячеславівна

(підпис)

Науковий керівник:
доцент кафедри інтелектуальних програмних
систем, кандидат фіз.-мат. наук
Шкільняк Оксана Степанівна

(підпис)

Засвідчую, що в цій роботі
немає запозичень з праць інших авторів
без відповідних посилань.
Студентка

(підпис)

КИЇВ 2026

РЕФЕРАТ

Обсяг роботи 34 сторінки, 4 ілюстрації, 3 таблиці, 29 джерел посилання.

API, DOCKER, PERIOD AFTER OPENING, POSTGRESQL, REACT NATIVE, TYPEORM, ЖИТТЄВИЙ ЦИКЛ, КЛІЄНТ-СЕРВЕРНА АРХІТЕКТУРА, КОСМЕТИЧНИЙ ЗАСІБ, МОБІЛЬНИЙ ЗАСТОСУНОК, ТЕРМІН ПРИДАТНОСТІ.

Об'єктом роботи є процес управління життєвим циклом косметичних засобів та автоматизації моніторингу безпеки їх використання. Предметом роботи є програмний засіб, який складається з мобільного застосунку та серверної backend-частини для контролю термінів придатності косметичних засобів.

Метою курсової роботи є розробка клієнт-серверної системи, яка дозволить користувачам вести облік косметичних засобів у використанні, відслідковувати їх термін придатності після відкриття (Period after opening), а також спостерігати за їх рейтингом у спільноті застосунку.

Методами розробки є компонентно-орієнтований підхід, об'єктно-реляційне моделювання даних, клієнт-серверна архітектура, методи динамічного обчислення часових інтервалів. Інструменти розробки: мобільна крос-платформа Ехро (React Native), TypeScript, Express, Node.js, PostgreSQL, TypeORM.

Результати роботи: розроблено клієнтський мобільний застосунок та серверний API для моніторингу життєвого циклу косметичних засобів, зберігання інформації про власні засоби користувача та оцінку популярності косметичних засобів у спільноті застосунку.

За методами розробки та інструментальними засобами робота виконувалась з дотриманням сучасних тенденцій розробки клієнт-серверних платформ.

Розроблений мобільний застосунок може застосовуватися споживачами у повсякденному житті для безпечного користування косметичними засобами.

Також можлива адаптація системи для використання професійними візажистами чи косметологами для обліку професійної косметики, що використовується для клієнтів.

ЗМІСТ

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАЧЕННЯ	5
ВСТУП	6
1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА АЛЬТЕРНАТИВНИХ РІШЕНЬ	9
1.1 АНАЛІЗ ІНДУСТРІЇ КРАСИ В ПИТАННІ БЕЗПЕЧНОГО СПОЖИВАННЯ	9
1.1.1 Життєвий цикл та специфіка споживання косметичних засобів	9
1.1.2 Теоретичні основи параметру РАО	11
1.2 ПРОБЛЕМИ УПРАВЛІННЯ ЖИТТЄВИМ ЦИКЛОМ КОСМЕТИЧНИХ ЗАСОБІВ	11
1.2.1 Забезпечення безпеки користувачів	11
1.2.2 Запобігання переспоживанню	12
1.3 АНАЛІЗ АЛЬТЕРНАТИВНИХ РІШЕНЬ	12
1.3.1 Огляд існуючих рішень	13
1.3.2 Аналіз технологічних підходів для розробки мобільних застосунків	14
2 ПРОЄКТУВАННЯ АРХІТЕКТУРИ ТА БАЗИ ДАНИХ СИСТЕМИ	16
2.1 ОБҐРУНТУВАННЯ ВИБОРУ АРХІТЕКТУРНОГО СТИЛЮ	16
2.1.1 Обґрунтування вибору клієнт-серверної моделі	16
2.1.2 Обґрунтування вибору архітектурного стилю REST API	17
2.2 ПРОЄКТУВАННЯ БАЗИ ДАНИХ	18
2.2.1 Концептуальна модель даних (ER-схема)	19
2.2.2 Цілісність та оптимізація даних	20
2.3 АРХІТЕКТУРА СЕРВЕРНОЇ BACKEND ЧАСТИНИ	21
2.3.1 Опис програмних шарів та їх взаємодії	21
2.3.2 Системні механізми та Middleware	22
2.4 ПРОЄКТУВАННЯ МОБІЛЬНОГО ЗАСТОСУНКУ	23
3 ПРОГРАМНА РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ МОБІЛЬНОГО ЗАСТОСУНКУ	24
3.2 ПРОГРАМНА РЕАЛІЗАЦІЯ СЕРВЕРНОЇ ЛОГІКИ	27
3.3 ТЕСТУВАННЯ МОБІЛЬНОГО ЗАСТОСУНКУ	28
ВИСНОВКИ	31
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ	32

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАЧЕННЯ

- API — (Application Programming Interface) Програмний інтерфейс додатка;
- CSS — (Cascading Style Sheets) Каскадні таблиці стилів;
- ER-діаграма — (Entity-Relationship diagram) Діаграма «сутність-зв'язок»;
- HTTP — (HyperText Transfer Protocol) Протокол передачі гіпертексту;
- JSON — (JavaScript Object Notation) Текстовий формат обміну даними;
- JWT — (JSON Web Token) Стандарт для створення токенів доступу;
- ORM — (Object-Relational Mapping) Об'єктно-реляційне відображення;
- PAO — (Period After Opening) Термін придатності після відкриття;
- REST — (Representational State Transfer) Архітектурний стиль взаємодії компонентів;
- SQL — (Structured Query Language) Мова структурованих запитів;
- UI — (User Interface) Інтерфейс користувача;
- UX — (User Experience) Користувацький досвід.

ВСТУП

Оцінка сучасного стану об'єкта розробки. Індустрія краси в сучасних умовах невинно розвивається та діджиталізується. Це помітно і в українському сегменті ринку. За аналітичним дослідженням [1], українська косметична галузь розвивається і в складних умовах повномасштабної війни, демонструючи високий розвиток в сегменті онлайн-продажів. Насичення ринку різними виробниками і легка доступність товарів, наприклад, через онлайн платформи, призводить до того, що кількість косметичних засобів у користуванні звичайного споживача зростає.

Разом з цим, виникає проблема ефективного використання косметичних продуктів та моніторингу терміну їх придатності. Ця проблема може призвести як і до медичних ризиків, так і до переспоживання, адже користувач може забувати про вже куплені доглядові засоби.

Актуальність роботи та підстави для її виконання. Створення мобільного застосунку для управління та моніторингу циклу косметичних засобів є актуальним завданням, адже прямо впливає на здоров'я користувача. Можливість візуально відслідковувати дотримання термінів використання РАО запобігає користуванню зіпсованою продукцією, в якій можуть розвиватися бактерії, окислятися компоненти тощо. Це призводить до втрати ефективності косметичного засобу, а в гіршому випадку – до подразнень шкіри, алергічних реакцій або навіть інфекцій.

Окрім цього, мобільний застосунок допомагає частково вирішити проблему переспоживання. Збереження всієї колекції доглядових засобів та сповіщення користувача про термін придатності, що закінчується, запобігає купівлі засобів-дублікатів та мотивує закінчувати продукт до кінця, а не викидати його.

Мета й завдання роботи. Метою курсової роботи є розробка мобільного застосунку для зручного та автоматизованого спостереження за терміном придатності косметичних засобів, а також отримання інформації про колекцію

наявних косметичних продуктів. Такий застосунок підвищує контроль за безпекою користування доглядовими продуктами.

Для досягнення цієї мети поставлено такі завдання:

- проаналізувати предметну область та визначити ключові параметри життєвого циклу косметичних засобів;
- спроектувати архітектуру клієнт-серверної системи з використанням реляційної бази даних;
- спроектувати та реалізувати API для взаємодії з frontend-частиною, а також для зберігання даних про користувача, його колекції косметичних засобів та інформацію про них;
- розробити frontend-частину для візуалізації даних.

Об'єкт, методи й засоби розроблення. Об'єктом роботи є процес управління життєвим циклом та моніторингу безпеки використання косметичних засобів.

Методи розроблення включають:

- клієнт-серверна архітектура для синхронізації даних;
- ORM для взаємодії з базою даних;
- компонентно-орієнтований підхід до побудови UI.

Технічний стек розроблення:

- Visual Studio Code як середовище розробки (IDE);
- TypeScript як мова програмування;
- Express.js як фреймворк для розробки API;
- React Native з використанням платформи Expo як фреймворк для розробки мобільного клієнту;
- PostgreSQL для творення бази даних;
- TypeORM для типізації запитів;
- Docker для контейнеризації бази даних;

- NativeWind та Gluestak UI для реалізації UI.

Можливі сфери застосування. Розроблений у ході виконання курсової роботи мобільний застосунок може бути використаний кінцевими споживачами для особистого користування, а також адаптований під професійне користування для обліку розхідних матеріалів.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА АЛЬТЕРНАТИВНИХ РІШЕНЬ

1.1 Аналіз індустрії краси в питанні безпечного споживання

Споживач щодня взаємодіє з індустрією краси, користуючись доглядовими засобами різного хімічного складу. В умовах стрімкого розвитку цієї галузі та збільшення асортименту, звичайному споживачу стає складніше контролювати якість косметичних продуктів. Предметна область цієї курсової роботи фокусується на перетині споживчої поведінки, хімічної безпеки продуктів та можливостей інформаційних технологій для автоматизації та полегшення контролю за цими процесами.

1.1.1 Життєвий цикл та специфіка споживання косметичних засобів

Життєвий цикл косметичного засобу з точки зору споживача – це певна послідовність станів, кожен з яких потребує особливих умов для зберігання та моніторингу часу використання.

Ці стани можна розподілити так:

- Придбання засобу. На цьому етапі користувач орієнтується на властивості засобу, а також на загальний термін придатності, зазначений на упаковці. Важливо зазначити, що на цьому етапі косметичний засіб перебуває в герметичному пакуванні;
- Пасивне зберігання. Коли користувач зберігає продукт до його відкриття. На цьому етапі важливо дотримуватися норм зберігання;
- Початок використання. Це момент, коли користувач відкриває косметичний засіб. Саме з цього моменту починається відлік РАО;

- Активне використання. Це найбільш тривалий етап, на якому продукт піддається зовнішньому впливу. На цьому етапі має відбуватися моніторинг закінчення терміну РАО;
- Завершення використання. На цьому етапі користувач або вичерпує косметичний засіб, або утилізує його, в тому числі й через збігу терміну придатності або зміну властивостей засобу.

Нижче на рисунку 1.1 наведено візуалізацію станів життєвого циклу, які автоматизуються у межах цієї курсової роботи.

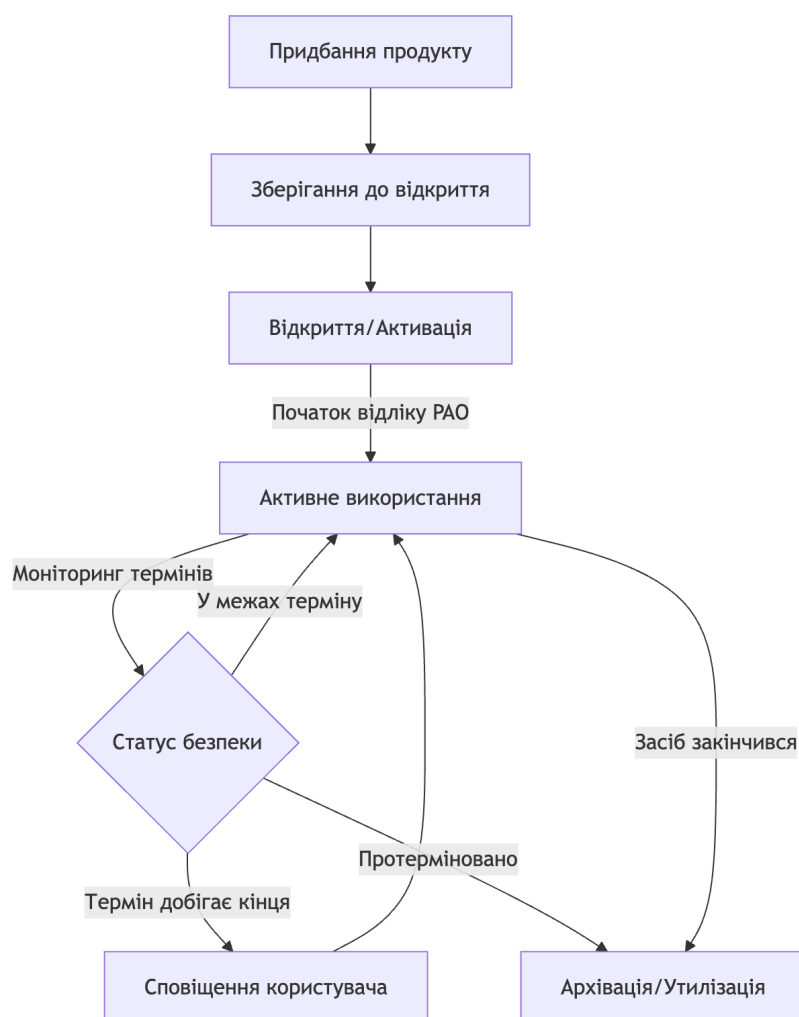


Рисунок 1.1 - Стани життєвого циклу косметичних засобів

1.1.2 Теоретичні основи параметру РАО

Для косметики у активному використанні ключовим показником безпеки використання є РАО. Цей показник відображає, протягом якого терміну після відкриття продукту можна безпечно його використовувати. Згідно з Директорією ЄС 1223/2009 [2], цей параметр є обов'язковим для косметичних засобів, загальний термін придатності яких перевищує 30 місяців.

Теоретична складність РАО полягає в тому, що він є “прихованим” для споживача. Замість загального терміну придатності (наприклад, Вжити до: 12.05), РАО - це лише інструкція з обчислення (наприклад, 12м).

1.2 Проблеми управління життєвим циклом косметичних засобів

1.2.1 Забезпечення безпеки користувачів

Згідно з медичними дослідженнями [3], використання доглядових засобів після завершення РАО може нести серйозну загрозу для здоров'я споживача.

Дослідження виявило такі важливі факти:

- до 90% косметичних засобів в активному використанні мають ознаки бактеріального або грибкового зараження;
- у протермінованих засобах консерванти можуть втрачати здатність боротися з патогенними бактеріями. Це може викликати не лише шкірні подразнення, а й інфекції очей, сечовивідних шляхів та навіть менінгіту у складних випадках;
- понад 90% опитаних споживачів продовжують використання косметичних засобів після закінчення РАО. Складність моніторингу дати

закінчення цього показника є основною причиною такої поведінки, споживачу просто складно згадати, коли засіб був відкритий.

Розроблений в ході курсової роботи мобільний застосунок вирішує цю проблему, надаючи користувачу зручну платформу для відстеження термінів придатності косметичних засобів.

1.2.2 Запобігання переспоживанню

З розширенням вибору в індустрії краси користувачу стає все легше здійснювати імпульсивні покупки, що може призвести до надмірного споживання. Завдяки візуалізації повної колекції косметичних засобів користувача розроблений застосунок допомагає боротися з:

- покупкою дублюючих засобів, адже користувач бачить всі наявні засоби в одному додатку та не купує новий продукт з категорії, яку вже має;
- нераціональним використанням продукту, адже розроблений панель із засобами, термін придатності яких скоро збігає, спонукає користувача використовувати саме його;
- зайвими відходами, які з'являються через забутий косметичний засіб, термін придатності якого вже сплив.

Окрім цього, запобігання переспоживанню позитивно впливає на персональний бюджет користувача.

1.3 Аналіз альтернативних рішень

Для того, щоб визначити прогалини в ніші подібних застосунків та розробити оптимальне рішення, необхідно провести аналіз вже наявних мобільних додатків зі схожими функціями.

1.3.1 Огляд існуючих рішень

На сьогодні ринок пропонує кілька категорій застосунків для управління косметичними засобами. Для аналізу було взято кілька популярних застосунків, які з'являються за релевантними запитами у магазині застосунків AppStore:

- Cosmetrack – дозволяє відслідковувати терміни РАО та всю колекцію користувача, але не має системи відгуків на косметичні засоби від користувача, не має соціальної складової, а також окремої панелі для косметичних засобів, термін яких закінчується [4];

- Beauty Vox - застосунок сфокусований на відстеженні дати виробництва косметичних засобів та не має логіки РАО. Створити власну колекцію доглядових продуктів можливо, але в безкоштовній версії функціонал обмежений [5].

- FeelinMySkin - загальний щоденник догляду за шкірою обличчя. Застосунок перевантажений різноманітними функціями, тому користувачу складно зібрати в одному місці всі доглядові засоби та спостерігати за термінами придатності [6].

Для зручного порівняння функціоналу популярних аналогів наведемо таблицю 1.1.

Таблиця 1.1 – Порівняльний аналіз можливостей існуючих рішень

Функціональність	Cosmetrack	Beauty Vox	FeelinMySkin
Автоматичний каталог (parsing)	Ні	Частково	Так
Логіка РАО	Так	Ні	Ні
Панель з продуктами, термін яких спливає	Ні	Ні	Ні

Продовження таблиці 1.1

Відгуки спільноти	Ні	Ні	Так
Алгоритм популярних продуктів на основі дій спільноти	Ні	Ні	Так

1.3.2 Аналіз технологічних підходів для розробки мобільних застосунків

Окрім аналізу наявних рішень, перед розробкою було досліджено різні технологічні підходи для розробки мобільних додатків. В розробці виділяють три основні парадигми розробки мобільних застосунків:

- Native Development передбачає написання окремого коду для кожної ОС (Swift/Objective-C для iOS [7] та Kotlin/Java для Android [8]). Розробка може бути ресурсозатратною, проте надає повний доступ до специфічного апаратного забезпечення обраної ОС;

- PWA є вебсайтами, що імітують мобільні застосунки. Вони є низькозатратними, проте мають обмежений доступ до функціоналу ОС (наприклад, до сповіщень iOS) та можуть програвати у продуктивності при складних застосунках [9];

- Кросплатформні фреймворки, наприклад React Native, дозволяють створювати нативні інтерфейси з однієї кодової бази. З цим фреймворком можна розробляти компонентну архітектуру, тобто перевикористовувати вже створені елементи. Це значно прискорює розробку [10].

Для наочного порівняння складемо таблицю 1.2.

Таблиця 1.2 – Порівняльний аналіз технологічних підходів до розробки

Критерій	Native Development	PWA	React Native
Продуктивність	Найвища	Низька	Висока
Швидкість розробки	Висока	Низька	Середня
Доступ до нативних API	Повний	Обмежений	Повний
Складність оновлень	Висока	Низька	Низька
Масштабованість	Висока	Середня	Висока
Надійність	Висока	Середня	Висока

За результатами проведеного аналізу було вирішено обрати кросплатформний підхід з використанням фреймворку React Native та платформи Ехро, забезпечуючи високу якість мобільного додатка при оптимізованих витратах ресурсів.

2 ПРОЄКТУВАННЯ АРХІТЕКТУРИ ТА БАЗИ ДАНИХ СИСТЕМИ

2.1 Обґрунтування вибору архітектурного стилю

Вибір архітектурного стилю є важливим рішенням. Це рішення визначає надійність, гнучкість, та здатність системи до масштабування. Для розроблення застосунку для моніторингу та управління життєвим циклом косметичних засобів було обрано клієнт-серверну архітектуру на базі архітектурного стилю REST. Таке поєднання є оптимальним для сучасних мобільних застосунків, адже вони потребують високої доступності даних та централізованого управління бізнес-логікою.

2.1.1 Обґрунтування вибору клієнт-серверної моделі

Вибір клієнт-серверної моделі замість розроблення автономного (standalone) локального застосунку базується на декількох критичних інженерних та користувацьких перевагах [11]:

- Забезпечення цілісності та централізації глобального каталогу.

Система передбачає наявність великої бази даних косметичних продуктів (брендів, описів, типових значень РАО). Зберігання такої бази безпосередньо у пам'яті мобільного пристрою є неефективним через значні обсяги даних та складність їхнього оновлення. Клієнт-серверна модель дозволяє створити «єдине джерело істини» (Single Source of Truth) на сервері, де каталог оновлюється централізовано та стає миттєво доступним для всіх клієнтів;

- Надійність та збереження персональних даних. Одним із недоліків проаналізованих аналогів є втрата даних при зміні або втраті смартфона. Завдяки

серверній частині, вся історія колекції користувача, дати відкриття засобів та відгуки зберігаються у хмарній базі даних PostgreSQL. Це гарантує безперервність життєвого циклу моніторингу незалежно від стану апаратного забезпечення клієнта;

- Оптимізація обчислювальних ресурсів пристрою. Сучасні мобільні пристрої мають обмежений ресурс батареї та процесорного часу. Важкі операції, такі як агрегація даних для алгоритму «Trending» (підрахунок популярності серед тисяч записів) або складні пошукові запити, делегуються серверу. Мобільний застосунок у цій схемі забезпечує високу швидкість відгуку інтерфейсу та економне енергоспоживання;

- Безпека та контроль доступу. Централізована модель дозволяє реалізувати надійну систему автентифікації. Чутлива інформація (наприклад, хеші паролів) ніколи не передається і не зберігається на клієнтській стороні в незахищеному вигляді, а доступ до API контролюється сервером.

2.1.2 Обґрунтування вибору архітектурного стилю REST API

Для організації взаємодії між клієнтом та сервером було обрано архітектурний стиль REST, який базується на використанні стандартних методів протоколу HTTP. Вибір на користь REST обумовлений наступними факторами:

- Відсутність стану (Statelessness) [12]. Згідно з принципами REST, сервер не зберігає інформацію про сесію клієнта між запитами. Кожен запит містить у собі повний контекст (наприклад, JWT-токен), необхідний для його виконання. Це критично для мобільних додатків, де інтернет-з'єднання може бути нестабільним: клієнт може легко відновити роботу після перепідключення до мережі, не потребуючи відновлення складних станів на сервері.

- Уніфікований інтерфейс та ресурсна орієнтованість. В архітектурі REST кожна сутність (Продукт, Засіб у колекції, Відгук) розглядається як окремий ресурс. Це дозволяє використовувати стандартні методи HTTP (GET — отримання, POST — створення, PUT — оновлення, DELETE — видалення), що робить структуру API передбачуваною, легкою для документування та подальшої підтримки.

- Ефективність передачі даних за допомогою JSON. Обмін даними відбувається у форматі JSON. Він є текстовим, легковаговим та легко читається як людиною, так і програмними засобами. Використання JSON мінімізує обсяг трафіку, що передається через мобільні мережі, та ідеально інтегрується зі стеком технологій Node.js та React Native.

- Масштабованість та незалежність шарів. REST дозволяє незалежно розвивати клієнтську та серверну частини. Наприклад, ми можемо повністю змінити внутрішню логіку роботи сервера або замінити базу даних, не вносячи змін у код мобільного застосунку, доки дотримується визначений контракт API.

Таким чином, комбінація клієнт-серверної моделі та REST API забезпечує створення стабільної, продуктивної та сучасної екосистеми для управління життєвим циклом косметичних засобів.

2.2 Проектування бази даних

База даних системи розроблена за реляційною моделлю та реалізована за допомогою СУБД PostgreSQL. Структура бази даних спроектована таким чином, щоб забезпечити цілісність інформації про продукти, високу швидкість пошуку та ефективне управління персональними колекціями користувачів.

2.2.1 Концептуальна модель даних (ER-схема)

Система базується на взаємодії п'яти основних сутностей, що відображають життєвий цикл косметичного засобу від каталогу до персональної полиці користувача:

- 1) User (Користувач): Центральний об'єкт автентифікації та персоналізації. Містить унікальний `email`, `password\_hash` (Bcrypt), налаштування інтерфейсу (`app\_theme`) та зовнішній ключ `image\_id` для аватара.
- 2) Product (Продукт): Глобальний каталог засобів. Слугує шаблоном для створення записів у колекціях. Поле `source\_status` (Enum: parsed/added_manual) дозволяє відрізнити верифіковані брендові позиції від записів, доданих користувачами вручну.
- 3) CollectionItem (Засіб у колекції): Сутність, що фіксує факт володіння конкретним екземпляром продукту. Зберігає динамічні параметри: `opened\_date` (дата відкриття), `rao` (період після відкриття в місяцях) та статус об'єкта (`active/archived`).
- 4) Review (Відгук): Інструмент зворотного зв'язку, що пов'язує досвід користувача з продуктом. Містить текстовий коментар та числовий рейтинг (1–5). Унікальний індекс `Unique(user\_id, product\_id)` гарантує відсутність дублюючих відгуків.
- 5) Image (Зображення): Спільний ресурс для зберігання метаданих медіа-файлів.

Для кращої візуалізації структури бази даних наведемо рисунок 2.1.

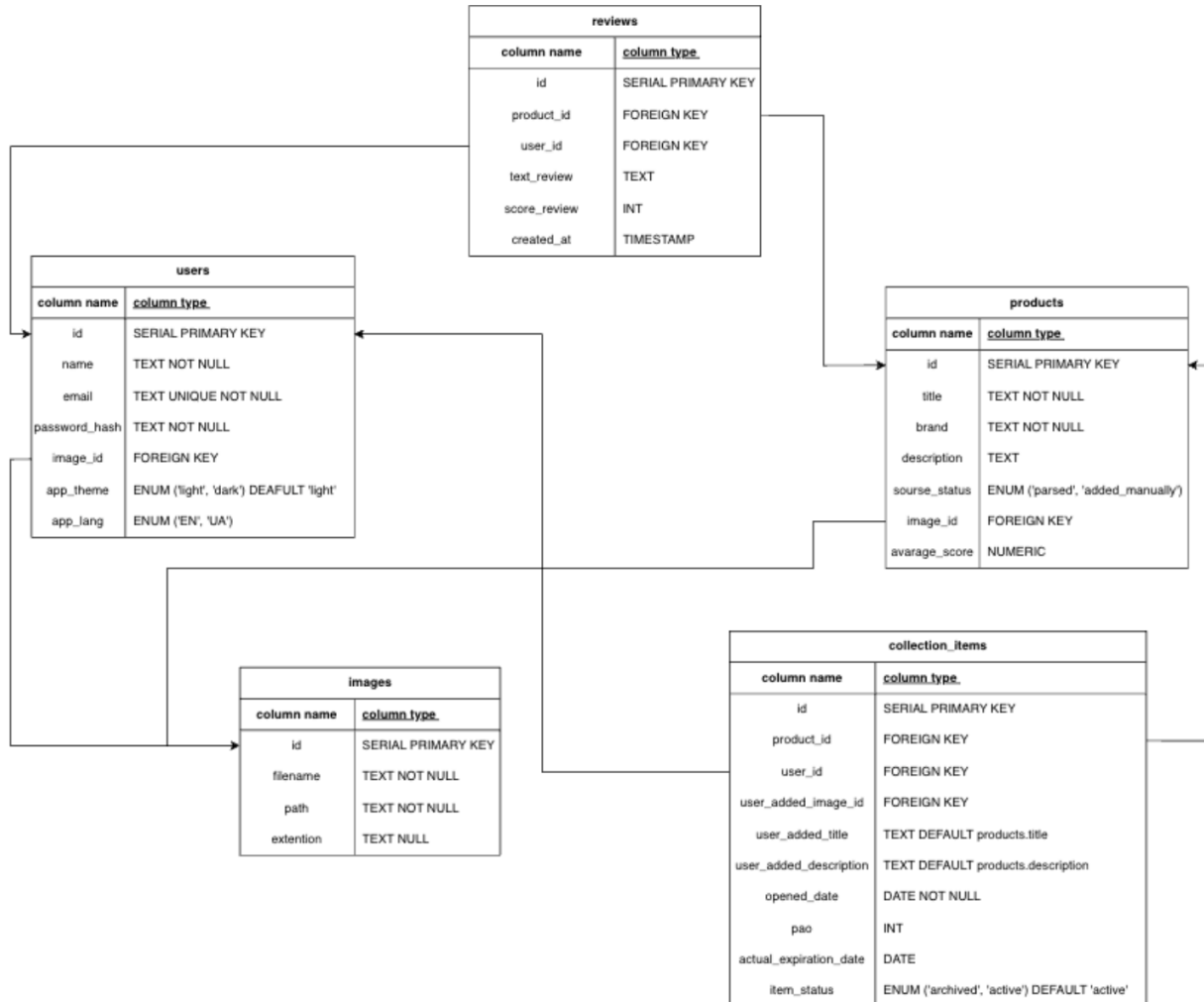


Рисунок 2.1 - ER модель бази даних

2.2.2 Цілісність та оптимізація даних

Для підтримки стабільності системи на рівні СУБД реалізовано наступні механізми:

- Каскадні операції: Використання 'ON DELETE CASCADE' для зв'язків 'User -> CollectionItem' та 'User -> Review'. Це гарантує повне очищення приватних даних при видаленні облікового запису користувача.

- Типізація через ENUM: Активне використання перелічень для статусів та причин архівації мінімізує обсяг збережених даних та повністю виключає можливість запису некоректних станів у базу.
- Індексація: Створення B-tree індексів за полями ``user_id`` та ``product_id`` для прискорення вибірок персональних колекцій та підрахунку рейтингів.
- Агрегація рейтингів: Для підвищення продуктивності системи при відображенні списків продуктів, середній бал (``average_score``) зберігається безпосередньо в таблиці ``products``. Оновлення цього значення відбувається автоматично при кожній зміні у таблиці ``reviews``, що забезпечує баланс між швидкістю читання даних та їхньою консистентністю.

2.3 Архітектура серверної backend частини

2.3.1 Опис програмних шарів та їх взаємодії

Серверна частина системи розроблена на платформі Node.js із використанням TypeScript. В основі архітектури лежить принцип багат шарового розподілу відповідальності [13], що дозволяє ізолювати логіку обробки HTTP-запитів від бізнес-правил та роботи з базою даних.

Процес обробки запиту проходить крізь чотири спеціалізовані шари:

- 1) Шар маршрутизації (Routes) визначає структуру API. Кожен ресурс має свій файл конфігурації. Саме тут підключаються механізми захисту, який перевіряє валідність JWT-токена перед допуском до контролера.
- 2) Шар контролерів (Controllers) відповідальний за парсинг вхідних даних (query params, body) та формування стандартизованих HTTP-відповідей.

3) Шар сервісів (Services) є місцем зосередження складної бізнес-логіки. Сервіси є "stateless" об'єктами. Прикладом є сервіс для відгуків, який окрім базових операцій, реалізує механізм автоматичного перерахунку середнього рейтингу продукту. Кожного разу, коли додається, змінюється або видаляється відгук, сервіс ініціює SQL-запит для оновлення значення в головній таблиці продуктів.

4) Шар сутностей (Entities/Persistence) реалізований через TypeORM. Кожен клас-сутність (User, Product тощо) є одночасно і схемою таблиці, і об'єктом у коді. Це забезпечує типобезпеку: помилка в назві поля буде виявлена ще на етапі компіляції проєкту.

2.3.2 Системні механізми та Middleware

Окрім бізнес-логіки, архітектура серверної частини включає кілька системних модулів. Одним із них є система автентифікації на базі JWT Strategy [14]. Сервер видає токен під час логіну, після чого перевіряє його підпис у кожному захищеному запиті. Такий підхід дозволяє легко масштабувати систему, оскільки повністю зникає необхідність синхронізації сесій між серверами.

За обробку медіа-даних, зокрема завантаження фотографій засобів та аватарів користувачів, відповідає інтегрований middleware Multer. Він автоматично виконує фільтрацію за типом файлу, дозволяючи завантажувати лише формати jpg та png. Після перевірки модуль зберігає файли у файлову систему, генеруючи для них унікальні імена, що ефективно запобігає колізіям та переповненню сховища.

Важливу роль в архітектурі також відіграють процеси валідації та загальна безпека застосунку. Абсолютно всі вхідні дані обов'язково проходять через відповідні шари перевірки перед тим, як система почне з ними працювати. Крім того,

взаємодія з базою даних побудована на використанні ORM Query Builder замість сирих рядкових запитів.

2.4 Проєктування мобільного застосунку

Архітектура застосунку побудована за допомогою React Native та платформи Ехро. Важливим аспектом проєктування стала організація внутрішньої структури застосунку. Для запобігання надмірному розростанню глобальних папок було обрано модульний підхід Feature-First. Згідно з цією парадигмою, кожна значуща функція застосунку виділяється у незалежну директорію, що містить React-компоненти, відповідальні виключно за рендеринг інтерфейсу та React-хуки, що реалізують інкапсульовану бізнес-логіку, запити до API та обробку станів. Такий розподіл дозволяє розробнику фокусуватися на конкретному функціоналі, не перемикаючись між десятками віддалених папок.

Навігація у застосунку побудована на базі Ехро Router, що використовує файлову систему маршрутизації. Це дозволяє проєктувати переходи між екранами згідно з ієрархією файлів. Структура навігації включає:

- Tab-Bar Layout: Головне меню для миттєвого перемикання між дашбордом, колекцією та формою пошуку продуктів.
- Stack Navigation: Забезпечує нативні переходи при відкритті деталей продукту або налаштувань профілю, підтримуючи звичні для користувачів жести повернення.

3 ПРОГРАМНА РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ МОБІЛЬНОГО ЗАСТОСУНКУ

3.1 Розробка клієнтської частини мобільного застосунку

Процес реалізації системи розпочався з розробки клієнтської частини, оскільки саме інтерфейс користувача визначає основні сценарії взаємодії та вимоги до структури даних. Мобільний застосунок побудовано на базі фреймворку React Native з використанням платформи Expo, що забезпечило швидкий цикл розробки та легку інтеграцію з нативними функціями пристроїв. Для забезпечення модульності та масштабованості коду було обрано компонентно-орієнтований підхід, де вся логіка відображення зосереджена в директорії `'components/'`. Кожен функціональний блок, такий як головний екран (`'MainScreen'`), колекція (`'CollectionScreen'`) чи профіль, має власну структуру, що включає головний компонент, типізацію TypeScript та кастомні хуки для бізнес-логіки. Особливу роль у структурі відіграє `'Expo Router'`, який реалізує файлову систему маршрутизації, дозволяючи організувати навігацію між екранами у декларативному стилі.

Використання NativeWind дозволило впровадити сучасний підхід до стилізації через класи Tailwind CSS, які під час компіляції трансформуються у високоефективні об'єкти StyleSheet. Дизайн-система додатково базується на компонентах Gluestack UI, що дозволило використовувати готові примітиви для побудови складних інтерфейсів з повною підтримкою TypeScript.

Одним із ключових завдань фронтенду стала реалізація зручного моніторингу термінів придатності. Для цього на головному екрані (`'MainScreen'`) було розроблено спеціалізований модуль `'ExpiredDashboard'`. Замість того, щоб змушувати користувача шукати протерміновані товари в загальному списку, цей дашборд автоматично формує горизонтальну карусель із засобів, термін придатності яких

спливає найближчим часом. Дані для цього блоку постачаються через кастомний хук `useMainScreenData`, який викликає відповідний метод API. На рівні компонента `ExpiredDashboard` реалізовано горизонтальний `FlatList`.

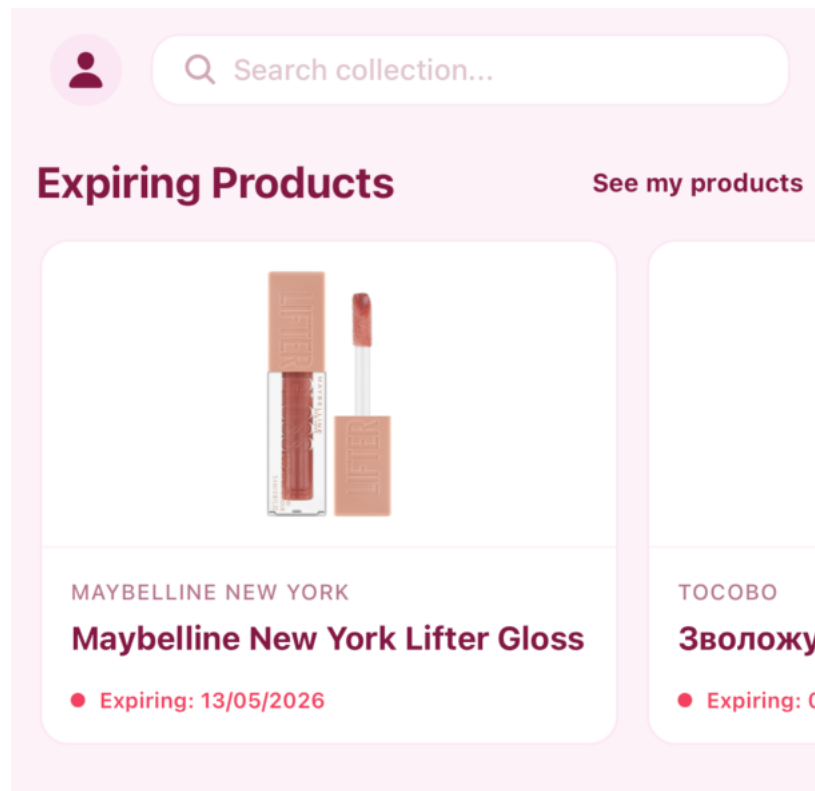


Рис. 3.1 - Реалізований модуль `ExpiredDashboard` в застосунку

Інша важлива функція для управління життєвим циклом косметичного засобу – архівування користувачем власного засобу із вказанням причини. інтелектуальної системи управління життєвим циклом косметики. Для цього було розроблено модуль архівування товарів, який дозволяє користувачу не просто видалити засіб, а перенести його в архів із зазначенням конкретної причини: «Закінчився» або «Вийшов термін придатності». Це дозволяє збирати статистику використання та допомагає користувачу аналізувати свою споживчу поведінку. На рівні інтерфейсу

це реалізовано через модальні вікна вибору причини та автоматичне оновлення статусу в базі даних.

Ще одним технічно складним етапом став модуль додавання нового продукту. Для покращення користувацького досвіду було впроваджено патерн Wizard (багатокрокова форма). Процес розділений на етапи: інтерактивний пошук по глобальній базі (з використанням оптимізації запитів), візуальне підтвердження через екран прев'ю та фінальна конфігурація персональних параметрів (дата відкриття, термін РАО). Логіка управління станом цієї форми інкапсульована в кастомному хуку `useAddProduct.ts`, який який синхронізує введені дані між кроками та виконує попередню валідацію перед відправкою через екземпляр Axios (`apiClient`).

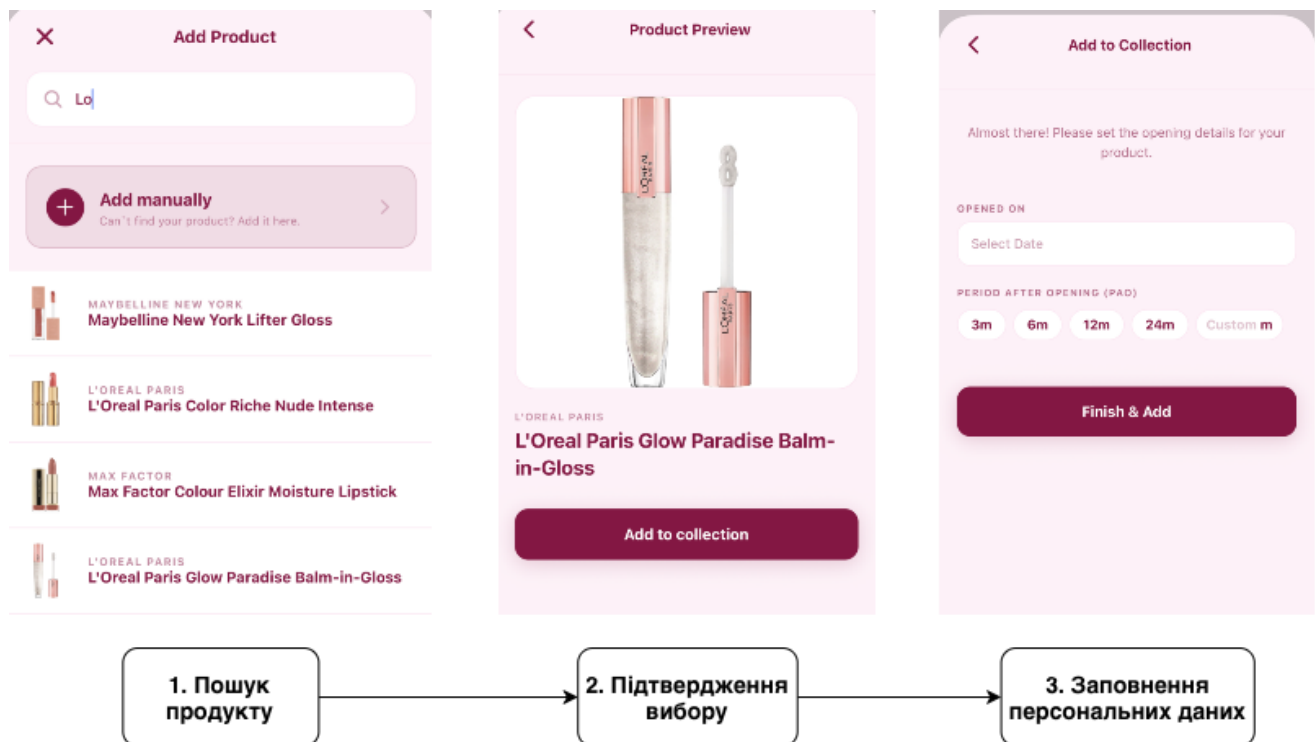


Рисунок 3.2 - Додавання продукту в мобільному додатка

3.2 Програмна реалізація серверної логіки

Серверна частина системи розроблена на платформі Node.js з використанням фреймворку Express.js та мови TypeScript, щоб забезпечити сувору типізацію на всіх рівнях обробки даних. Основною метою розробки backend частини було створення стабільного RESTful API, здатного керувати каталогом косметичних засобів та забезпечувати безпеку персональних профілів. Для реалізації об'єктно-реляційного відображення (ORM) обрано бібліотеку TypeORM, яка автоматизує взаємодію з PostgreSQL.

Для автоматизації наповнення бази даних було розроблено спеціалізований скрипт `parse-products.js`, який виконує інтеграцію з популярним маркетплейсом косметики (Makeup) [15]. Скрипт через HTTP-запити до API маркетплейсу отримує дані про товари, автоматично створює записи в таблиці `Image` та наповнює таблицю `Product` зі статусом `parsed`. Це дозволило сформувати базовий каталог із сотень товарів, завдяки чому користувачам не потрібно вводити дані вручну для більшості засобів.

Архітектура бази даних, реалізована через TypeORM, підтримує розгалужену систему статусів походження даних (`parsed` та `added_manually`) та станів використання (`active`, `archived`). Продукти зі статусом `parsed` вважаються частиною глобального стабільного каталогу і не видаляються системою. Натомість, продукти зі статусом `added_manually` — це записи, створені конкретними користувачами. Для запобігання накопиченню унікальних або некоректних записів, у `CollectionService` реалізовано логіку очищення: при видаленні користувачем такого засобу, система перевіряє кількість посилань на нього. Якщо він більше ніким не використовується, сервер ініціює видалення самого продукту та його зображення.

У випадку архівування, система додатково фіксує `archive_reason` та `expiry_relation`, що дозволяє відстежувати, чи був засіб використаний вчасно, або ж він був викинутий через протермінування.

Одним із найбільш технологічних аспектів став механізм денормалізації даних для мінімізації «важких» JOIN-запитів. Для швидкого відображення рейтингів середній бал (`averageScore`) зберігається безпосередньо в таблиці `Product`. У `ReviewService` реалізовано приватний метод `updateProductAverageScore`, який автоматично викликається після кожного створення, редагування або видалення відгуку. Він виконує агрегаційний запит до таблиці `Review` та миттєво оновлює головний запис продукту.

Безпека системи базується на використанні JWT (JSON Web Tokens) та middleware-підходу. Процес авторизації розділений на два етапи: хешування пароля при реєстрації за допомогою `bcrypt` (з використанням `salt`) та генерація підписаного токена при вході. Кожен запит до захищених маршрутів (наприклад, `/api/collection`) проходить через `authMiddleware`, який витягує токен із заголовка `Authorization`, верифікує його за допомогою секретного ключа та додає ідентифікатор `userId` до об'єкта запиту `Express.Request`. Для обробки завантажень зображень інтегровано бібліотеку `multer`, яка сконфігурована на збереження файлів з унікальними іменами (через `Date.now()`) та фільтрацію по MIME-типах (тільки `image/jpeg` та `image/png`). Така архітектура забезпечує високу швидкість відповіді сервера та надійний захист конфіденційних даних користувачів.

3.3 Тестування мобільного застосунку

Заключним етапом розробки стала комплексна верифікація системи. Тестування проводилося ітераційно, охоплюючи математичну логіку обчислень,

клієнт-серверу взаємодію та UX-показники. Основним об'єктом перевірки стала утиліта `date.ts`, яка відповідає за розрахунок дати протермінування. Оскільки косметичні засоби можуть мати різні терміни (РАО), було важливо впевнитися, що алгоритм коректно обробляє високосні роки та переходи між календарними періодами. Результати верифікації з використанням межових значень підтвердили надійність програмної реалізації.

Таблиця 3.1 - Тестування обрахунку кінцевої дати споживання продукту в мобільному затосунку

Тестовий сценарій	Дата відкриття	РАО (міс.)	Очікуваний результат	Статус
Базовий розрахунок	01.01.2024	12	01.01.25	Успішно
Перехід через рік	15.10.2023	6	15.04.2024	Успішно
Високосний рік	31.01.2024	1	29.02.2024	Успішно

Під час функціонального та інтеграційного тестування перевірялася працездатність усіх модулів системи без виключення. Тестування проводилося у два етапи для забезпечення максимальної достовірності результатів: на емуляторі iOS (симулятор Xcode) для перевірки базової логіки, навігації та коректності верстки на різних поколіннях iPhone, а також на фізичному пристрої під управлінням ОС iOS для оцінки реальної швидкодії та стабільності роботи з апаратними модулями (камера для фото товарів, локальне сховище для кешування).

На обох середовищах було детально протестовано повний цикл роботи користувача: реєстрація та вхід, пошук у глобальному каталозі, додавання засобів

через багатокрокову форму, налаштування персональних термінів придатності, написання відгуків та процес архівування з вибором причини.

Результати вимірювань через React Native Performance на фізичному пристрої підтвердили високу продуктивність: час рендерингу головного екрану не перевищує 400мс. Таким чином, комплексне тестування підтвердило повну функціональну готовність та високу якість реалізованої системи.

ВИСНОВКИ

У ході виконання даної курсової роботи було успішно розроблено мобільний застосунок «My Cosmetic Tracker» та відповідну серверну частину для автоматизації моніторингу термінів придатності та управління персональною колекцією косметичних засобів.

У ході розробки було виконано такі основні завдання:

- Проведено ґрунтовний аналіз предметної області;
- Спроектовано та реалізовано клієнт-серверну архітектуру, яка базується на сучасному стеку технологій TypeScript;
- Створено інтелектуальний мобільний інтерфейс на базі React Native і Ехро;
- Реалізовано складну бізнес-логіку управління життєвим циклом даних;
- Розроблено та впроваджено алгоритм розрахунку термінів придатності, який стійкий до календарних особливостей;
- Проведено комплексне тестування та верифікацію результатів на симуляторі iOS та реальному фізичному пристрої під управлінням iOS.

Результати роботи мають високу практичну цінність, оскільки розроблений застосунок є завершеним програмним продуктом, готовим до реальної експлуатації. Впровадження системи дозволяє не лише автоматизувати рутинний процес контролю термінів придатності, а й формувати культуру свідомого та безпечного споживання косметичних засобів. Таким чином, усі поставлені завдання виконано в повному обсязі, а мета курсової роботи вважається досягнутою.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Аналітичний звіт: Стан косметичної галузі в умовах повномасштабної війни [Електронний ресурс] / Дія.Бізнес – Режим доступу до ресурсу: <https://business.diia.gov.ua/analytics/research/Analysis-state-cosmetic-industry-conditions-full-scale-war>
2. Regulation (EC) No 1223/2009 of the European Parliament and of the Council of 30 November 2009 on cosmetic products [Електронний ресурс] / Official Journal of the European Union – Режим доступу до ресурсу: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32009R1223>
3. S. S. Kim et al. Cosmetic-related adverse events and consumer behavior: A cross-sectional study [Електронний ресурс] / National Center for Biotechnology Information – Режим доступу до ресурсу: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12474526/>
4. Cosmetrack - Cosmetics Tracker on the App Store [Електронний ресурс] / App Store Preview – Режим доступу до ресурсу: <https://apps.apple.com/app/cosmetrack-cosmetics-tracker/id6499244151>
5. Beauty Box: Cosmetic Checker on the App Store [Електронний ресурс] / App Store Preview – Режим доступу до ресурсу: <https://apps.apple.com/app/beauty-box-cosmetic-checker/id6478144517>
6. FeelinMySkin: Skincare Routine Tracker [Електронний ресурс] / App Store Preview – Режим доступу до ресурсу: <https://apps.apple.com/ua/app/feelinmyskin-skincare-routine/id1484551717>
7. Documentation for Apple Developers [Електронний ресурс] / Apple Developer – Режим доступу до ресурсу: <https://developer.apple.com/documentation/>
8. Android Developers Documentation [Електронний ресурс] / Google Developers – Режим доступу до ресурсу: <https://developer.android.com/docs>
9. Progressive Web Apps (PWAs) [Електронний ресурс] / MDN Web Docs – Режим доступу до ресурсу: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

10. React Native: Learn once, write anywhere [Електронний ресурс] / Meta Platforms, Inc. – Режим доступу до ресурсу: <https://reactnative.dev/>
11. Martin Fowler. Patterns of Enterprise Application Architecture [Електронний ресурс] / Martin Fowler – Режим доступу до ресурсу: <https://martinfowler.com/eaCatalog/>
12. Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures. [Електронний ресурс] / University of California, Irvine. – 2000. – Режим доступу до ресурсу: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
13. Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. – Prentice Hall, 2017. – 432 p.
14. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT). RFC 7519. [Електронний ресурс] / IETF Datatracker. – 2015. – Режим доступу до ресурсу: <https://datatracker.ietf.org/doc/html/rfc7519>
15. Інтернет-магазин парфумерії та косметики MAKEUP [Електронний ресурс] / MAKEUP – Режим доступу до ресурсу: <https://makeup.com.ua/ua/>
16. Express: Node.js web application framework [Електронний ресурс] / Express documentation – Режим доступу до ресурсу: <https://expressjs.com/>
17. Expo Documentation: Create amazing apps with React Native [Електронний ресурс] / Expo – Режим доступу до ресурсу: <https://docs.expo.dev/>
18. Gluestack UI: Fast and accessible components for React and React Native [Електронний ресурс] / Gluestack – Режим доступу до ресурсу: <https://gluestack.io/>
19. NativeWind: Tailwind CSS for React Native [Електронний ресурс] / NativeWind Documentation – Режим доступу до ресурсу: <https://www.nativewind.dev/>
20. Node.js: JavaScript Runtime [Електронний ресурс] / Node.js Foundation – Режим доступу до ресурсу: <https://nodejs.org/en/about>
21. PostgreSQL: The World's Most Advanced Open Source Relational Database [Електронний ресурс] / PostgreSQL Global Development Group – Режим доступу до ресурсу: <https://www.postgresql.org/>

22. The State of Developer Ecosystem 2022 [Електронний ресурс] / JetBrains – Режим доступу до ресурсу: <https://www.jetbrains.com/lp/devecosystem-2022/>
23. TypeORM: Amazing ORM for TypeScript and JavaScript [Електронний ресурс] / TypeORM Documentation – Режим доступу до ресурсу: <https://typeorm.io/>
24. TypeScript: JavaScript with syntax for types [Електронний ресурс] / Microsoft – Режим доступу до ресурсу: <https://www.typescriptlang.org/>
25. Про захист прав споживачів: Закон України від 12.05.1991 № 1023-XII [Електронний ресурс] / Верховна Рада України – Режим доступу до ресурсу: <https://zakon.rada.gov.ua/laws/show/1023-12>
26. Про затвердження Технічного регламенту на косметичну продукцію: Постанова Кабінету Міністрів України від 20.01.2021 № 65 [Електронний ресурс] / Кабінет Міністрів України – Режим доступу до ресурсу: <https://zakon.rada.gov.ua/laws/show/65-2021-%D0%BF>
27. Docker Documentation. [Електронний ресурс] / Docker Inc. – Режим доступу до ресурсу: <https://docs.docker.com/>
28. TypeScript Language Specification. [Електронний ресурс] / Microsoft. – Режим доступу до ресурсу: <https://www.typescriptlang.org/docs/>
29. Human Interface Guidelines. [Електронний ресурс] / Apple Developer. – Режим доступу до ресурсу: <https://developer.apple.com/design/human-interface-guidelines/>